

Evaluating Stalwart Mail Server for IRON ID Email

Why Stalwart Outperforms Traditional Mail Stacks and Desktop Clients (Bilingual EN / FR)

Doc Ref: ARC-2026-EML-01
Date: September 2026
Classification: Internal
Confidential

Executive Summary: IRON ID Email is defined not as an isolated mailbox, but as an organizational extension of *IRON ID Auth* and *IRON ID Workspace*. While Mozilla Thunderbird is strictly a local desktop reader, **Stalwart Mail Server** delivers a modern, single-binary, memory-safe backend natively equipped with JMAP, OIDC, and S3-compatible sovereign storage.

Résumé Exécutif : *IRON ID Email n'est pas une simple messagerie isolée, mais une extension directe d'IRON ID Auth et Workspace. Tandis que Thunderbird est un client lourd poste utilisateur, Stalwart fournit un moteur serveur écrit en Rust, sécurisé en mémoire, intégrant nativement JMAP, OIDC et le stockage souverain compatible S3.*

1. STRUCTURAL CLARIFICATION: SERVER VS. CLIENT

Clarification Structurelle : Serveur vs. Client

Dimension	Mozilla Thunderbird	Stalwart Mail Server
Role / Rôle	Desktop / Mobile Client (Lecteur local) CLIENT ONLY	All-in-one Mail Server Engine (MTA, MDA, Storage) CORE ENGINE
Execution / Exécution	End-user PCs (Windows, macOS, Linux, Android)	Central sovereign Linux servers, clusters, or K8s nodes
Multi-Tenancy	Single user connecting to personal accounts	Multi-tenant SaaS hosting thousands of sovereign domains
Suitability for IRON ID	Cannot be used as backend infrastructure. Fits only as an optional desktop connector for end-users.	Optimal backend foundation powering the custom webmail module embedded inside Workspace.

2. STRATEGIC ADVANTAGES FOR IRON ID

Avantages Stratégiques pour IRON ID

1. Native JMAP vs. Legacy IMAP

Traditional IMAP requires stateful connections and complex parsing proxies for web applications. Stalwart natively speaks JMAP (RFC 8620/8621), allowing the custom IRON ID Workspace UI to query mailboxes and attachments using clean JSON/HTTPS.

Le protocole JMAP élimine la complexité d'IMAP en offrant des API REST/JSON directes, idéales pour le Webmail React d'IRON ID Workspace.

2. Rust Architecture & Memory Safety

Unlike C-based stacks (Postfix, Dovecot, Exim) susceptible to memory corruption vulnerabilities, Stalwart is written in 100% safe Rust. It collapses MTA, IMAP, JMAP, and spam filters into a single hyper-efficient binary.

Écrit en Rust, Stalwart élimine les failles de mémoire et regroupe MTA, IMAP et antispam en un binaire unique et sobre en ressources.

3. Seamless IRON ID Auth Integration

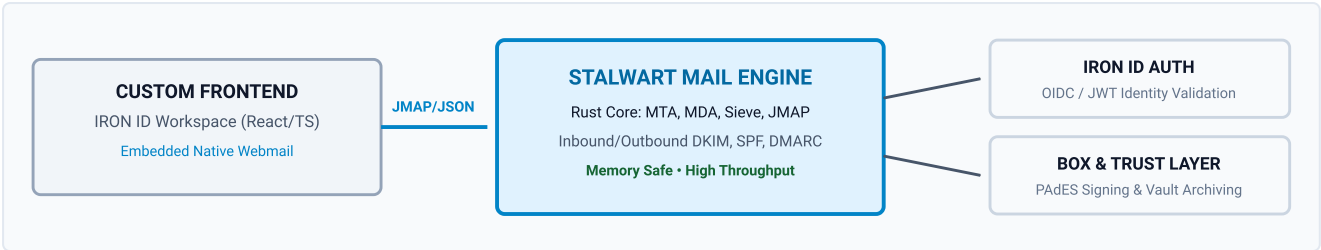
Stalwart natively supports OpenID Connect (OIDC) and OAuth 2.0. Users authenticate once via IRON ID Auth, passing standard JWT bearer tokens to the mail engine without managing separate email passwords.

4. Sovereign Data Architecture

Mail blobs can be stored in local S3-compatible clusters (MinIO, Ceph) and metadata in PostgreSQL, honoring the 3 IRON ID deployment models: National Hosting, Private Infra, and Isolated air-gapped zones.

3. ARCHITECTURAL TOPOLOGY: CUSTOM UI + STALWART

Topologie d'Architecture : UI Personnalisée + Stalwart



4. BACKEND BENCHMARK: STALWART VS. TRADITIONAL STACKS

Comparatif Serveur : Stalwart vs. Piles Traditionnelles

Evaluation Metric	Legacy Stack (Postfix + Dovecot)	Stalwart Mail Server
Architecture / Footprint	Fragmented: 4–6 daemons, heavy RAM usage COMPLEX	Single unified Rust binary; ultra-light RAM STREAMLINED
API Protocol for UI	Requires IMAP socket bridges or CalDAV sync	Native JMAP (JSON Meta Application Protocol)
Security Audit Profile	Legacy C code base; vulnerable to buffer overflows	100% Safe Rust; zero buffer overflow exposure
Sovereign Cloud Readiness	Complex POSIX volume mounting across pods	Native S3-compatible backend (MinIO/Ceph) & PGSQL

5. ACTION PLAN & ROADMAP

Plan d'Action et Feuille de Route

Phase 1: Foundation & Identity Binding

Deploy Stalwart in containerized staging. Configure OIDC client to validate Bearer tokens generated by IRON ID Auth. Provision tenant domains and mailboxes automatically via Stalwart REST management API.

Phase 1 : Socle et Liaison d'Identité – Déploiement conteneurisé, liaison OIDC avec IRON ID Auth et automatisation du provisioning des comptes par API.

Phase 2: Custom Workspace Webmail UI

Develop the React/TS mail module using standard JMAP endpoints (`Email/query`, `Email/get`, `EmailSubmission/set`). Add DOMPurify email body sanitization and real-time inbox synchronization via Server-Sent Events.

Phase 2 : Interface Webmail Dédiée – Développement du module React via JMAP, filtrage sécurisé du HTML et synchronisation temps réel par flux SSE.

Phase 3: Ecosystem Interoperability

Implement "1-Click Sign with Trust" for PDF attachments and "Evidentiary Archive to Box" with SHA-256 integrity sealing. Support Thunderbird and native mobile apps via standard IMAP/OAuth2 credentials.

Phase 3 : Intégration Écosystème – Boutons d'export vers Box, signature électronique certifiée et support des clients externes (Thunderbird, iOS, Android).

Phase 4: Sovereign Deployment Profiles

Validate packaging for all three client hosting topologies: National Multi-tenant hosting, Private client infrastructure, and air-gapped Isolated networks for governmental institutions.

Phase 4 : Modèles de Souveraineté – Validation du packaging sur les trois modes d'hébergement : National, Privé sur site, et Réseau Isolé (air-gapped).